The AI Collaboration Model I

The Documentation Duality standard and three-tier skill architecture represent an empirical bet: that structured, machine-readable documentation measurably improves AI agent performance in research codebases. This section reports our key observations.

The documentation investment creates a positive feedback loop: as agents produce higher-quality outputs from structured context, developers maintain that documentation, which in turn improves future interactions [@lau2025aicoding]. We observed this concretely during `template/` development—each module's `SKILL.md` was refined through iterative AI-assisted generation, serving as both input prompt and output validator.

The AI Collaboration Model II

The SKILL.md layer, with its MCP-aligned YAML frontmatter [@anthropic2024mcp], provides a critical bridge to the agentic software paradigm. Lu et al.'s AI Scientist [@lu2024aiscientist] demonstrates end-to-end autonomous research; Wang et al.'s OpenHands [@wang2024opendevin] achieved 53% on SWE-Bench Verified [@jimenez2024swebench]—the first open-source system to exceed 50%. These systems require structured, protocol-aligned tool inventories to navigate unfamiliar codebases. An OpenHands-class agent navigating template/ reads CLAUDE.md for global constraints, scans AGENTS.md for module surfaces, and invokes capabilities via SKILL.md without hallucinating function signatures. All 23 infrastructure modules carry AGENTS.md and README.md; all additionally carry SKILL.md, ensuring no documentation blind spots.

The AI Collaboration Model III

This three-tier model is, to our knowledge, novel in the research software engineering literature. The scale of the investment—approximately 170 documentation files across `docs/`, `AGENTS.md`/`README.md` pairs, and per-module skill descriptors—represents a deliberate commitment to machine-readable context that reduces hallucination surface area.

The Learning Curve I

The Thin Orchestrator pattern imposes a cognitive overhead on researchers accustomed to writing monolithic scripts. The requirement to factor all logic into `src/` modules and use scripts only as stateless wiring introduces an additional layer of indirection. We mitigate this through:

1. **Template exemplars**: `template_code_project` ships minimal optimization commentary; `template_prose_projec t` and `template_autoresearch_project` broaden narrative + retrieval scaffolding.
2. **Documentation Duality**: Every directory has both `README.md` (for humans) and `AGENTS.md` (for AI collaborators), reducing the cost of navigation.
3. **Interactive orchestrator**: `run.sh` provides a TUI menu that abstracts pipeline complexity.

The Learning Curve II

4. **Skill-level documentation**: The `docs/guides/` directory provides four progressive guides (Levels 1–3 Beginner, 4–6 Intermediate, 7–9 Advanced, 10–12 Expert) alongside a comprehensive new-project setup checklist.

Limitations I

- **LaTeX dependency**: The rendering pipeline requires a full TeX distribution (TeX Live or MiKTeX), which is a 4–6 GB install. This is the largest single dependency and is a barrier for researchers without system-level package management access.
- **Python-centric**: The infrastructure layer is Python-only. Projects in other languages can use the rendering and steganography stages but cannot leverage the `scientific` or `validation` modules.
- **Single-machine**: The pipeline runs locally. Distributed execution (e.g., across a compute cluster) is not natively supported, a gap where Snakemake, Nextflow, and CWL have clear superiority.

Limitations II

- **Steganographic robustness**: Alpha-channel overlays are stripped by aggressive PDF optimization tools (e.g., `qpdf --optimize`). QR codes are visible and removable. The current system provides *tamper detection* (via SHA-256 hashing) but not *non-repudiation* in the cryptographic sense—it lacks private-key digital signatures. An attacker with access to the source code could reproduce the watermark without having run the original pipeline.
- **Test duration**: The Zero-Mock policy increases test execution time from sub-second (mocked) to multi-minute (real) for the full infrastructure suite. This is acceptable for research workflows but may not suit continuous deployment scenarios.

Limitations III

- **AI-native writing tools**: `template/` does not include an integrated AI writing assistant comparable to Overleaf's Copilot features or OpenAI Prism's GPT-5.2 context-aware editing. The `infrastructure.llm` module provides LLM review as a pipeline stage but not as an interactive writing environment.